Hes·so

TSM-MachLeData

Stock Price Prediction using Machine Learning

20.10.2025

Justin Ferrara
Luca Da Silva
Rafael Cardoso
Thomas Maksimous
HES-SO Master
-

Table of Contents

1. Introduction 3
2. Advanced Topic - Data Drift 4
3. Project Scoping 4
   3.1. Objectives 4
   3.2. Requirements 5
   3.3. Project Constraints and Associated Risks 5
4. Data Engineering 5
   4.1. Data Acquisition 5
   4.2. Data Preprocessing 6
   4.3. Data Storage 6
5. ML Model Development 6
   5.1. Feature Engineering 6
   5.2. Model Training and Validation 6
   5.3. Model Evaluation 6
6. Deployment 7
   6.1. CI/CD Pipeline 7
   6.2. Infrastructure 8
   6.3. Scalability 8
7. Monitoring 8
   7.1. Data Drift Detection 8
   7.2. Continuous Learning 8
8. Conclusion 9

1. Introduction

The objective is to create a simple predictive market model. The model would take as input the stock closing price for each month, with the goal of predicting the closing prices for the next 5 days. The dataset would consist of the past year's data for the top 100 stocks of a stock market index.

As an advanced MLOps problem, we will study the data drift in our results. The goal is to fine-tune our model every week using the new data from the previous week. We will then compare the performance of our models over time. The objective here is to observe whether the performance of older models degrades over time compared to their updated fine-tuned versions. (Model drift)

2. Advanced Topic - Data Drift

In machine learning systems, **model drift** (also known as concept drift) refers to the degradation of a model's predictive performance over time due to changes in the underlying data distribution or relationships between input and output variables. This phenomenon occurs because the model was trained on past data that may no longer represent current real-world conditions. In financial forecasting, for instance, market trends, investor behavior, or economic indicators may evolve, causing previously learned patterns to become outdated. Detecting and managing model drift is therefore critical to maintaining accuracy and reliability in predictive systems.

One key source of model drift is **data drift**, which occurs when the statistical properties of input features (the X variables) change over time. For example, if stock trading volumes or volatility levels shift due to external economic events, the distribution of input data no longer aligns with what the model originally learned. Data drift can be measured through statistical tests such as the Kolmogorov-Smirnov test or by comparing feature distributions between training and production data. Regular monitoring allows to identify when the input data environment has shifted significantly, prompting the need for model retraining or adjustment.

A related but distinct phenomenon is **feature drift**, which occurs when the relevance or predictive power of certain input features changes relative to others. For example, a feature like « daily trading volume » might become less predictive of price movement if market liquidity conditions change. Feature drift affects model interpretability and can mislead the model's weighting of inputs, even if overall data distributions appear stable. Detecting feature drift often involves monitoring metrics like feature importance or mutual information across time.

Finally, **label drift** (or target drift) refers to a change in the distribution of the target variable (Y). In stock prediction, this could occur if the overall market sentiment changes and the expected returns follow a different trend than before. Label drift can appear independently of input changes. For instance, when a company changes its dividend policy or experiences sudden external shocks. Continuous evaluation on recent labeled data helps detect this drift and guides the decision to retrain or recalibrate the model.

Together, these forms of drift—model, data, feature, and label, highlight the importance of dynamic monitoring and adaptation in MLOps. Properly managing them ensures that machine learning models remain accurate, fair, and robust in evolving real-world environments.

TODO more references and go deeper

3. Project Scoping

The project aims to develop a machine learning model to predict stock opening prices, while implementing a comprehensive MLOps lifecycle to ensure the model's robustness and adaptability over time.

3.1. Objectives

The objective of this project is to predict the opening stock value of different companies using machine learning techniques. The purpose is to provide useful insights to investors, helping them make more informed and data-driven investment decisions. The key problem to solve is the accurate prediction of stock prices, which varies due to market volatility and numerous influencing factors.

The advanced MLOps topic covered in this project will be Model Drift detection and mitigation, more specifically the Data Drift and Feature Drift aspects. In this context, we will monitor how changes in the input data distribution over time affect the model's performance. The main goal is to maintain the

model's reliability and adaptability, ensuring it remains at its best performance level even as market conditions evolve.

3.2. Requirements

This project will require the following resources:

- Financial datasets containing daily stock data (open, close, high, low, volume, etc.) with sufficient historical depth.
- Development environment for data science and MLOps:
  - Python (pandas, scikit-learn, TensorFlow/PyTorch)
  - DVC or MLflow for experiment tracking and versioning
  - Docker and GitHub Actions for CI/CD
  - Cloud or local deployment environment (e.g., Google Cloud, Azure, or Kubernetes cluster)

Monitoring tools to track model drift (e.g., Evidently AI, Prometheus + Grafana, or custom scripts).

3.3. Project Constraints and Associated Risks

The project operates under multiple constraints and risks that could influence its success. Time limitations may restrict the extent of model experimentation and deployment testing, while limited financial domain expertise could block accurate interpretation of market-related outcomes. Additionally, access to reliable financial data may be limited or require paid sources, and the technical complexity of implementing MLOps processes such as continuous integration, monitoring, and retraining adds further challenges.

These constraints comes with risks including potential costs associated with cloud technologies and APIs, the inherent instability of models due to unpredictable market dynamics, and the possibility of overfitting, which could reduce the model's ability to generalize effectively to unseen market conditions.

4. Data Engineering

The data engineering phase is crucial for the success of the stock price prediction project. It involves several key steps to ensure that the data is properly collected, processed, and stored for effective use in machine learning models.

4.1. Data Acquisition

To acquire the necessary data for stock price prediction, we used the https://www.nasdaq.com website to download historical stock data for 20 stocks in the NASDAQ index. The data collected is in CSV format, and contains columns such as Date, Close, Open, High, Low, and Volume for the past 5 years. The following companies were selected for this project:

- Apple Inc. (AAPL)
- AMD Inc. (AMD)
- Amazon.com, Inc. (AMZN)
- ASML Holding N.V. (ASML)
- Broadcom Inc. (AVGO)
- Cisco Systems, Inc. (CSCO)
- Alphabet Inc. (GOOGL)
- International Business Machines Corporation (IBM)
- Facebook, Inc. (META)
- Microsoft Corporation (MSFT)
- Micron Technology, Inc. (MU)

- Netflix, Inc. (NFLX)
- NVIDIA Corporation (NVDA)
- Oracle Corporation (ORCL)
- Palantir Technologies Inc. (PLTR)
- Qualcomm Incorporated (QCOM)
- Starbucks Corporation (SBUX)
- Tencent Holdings Ltd. (TCEHY)
- Tesla, Inc. (TSLA)
- Taiwan Semiconductor Manufacturing Company Ltd. (TSM)

4.2. Data Preprocessing

The preprocessing steps involved cleaning the data and transforming the data into a suitable format for modeling. We performed the following preprocessing tasks:

- Checking for missing values and handling them appropriately (e.g., imputation or removal).
- Converting the closing prices to float data variables, representing the variation in percent from the previous closing price.
- Normalizing the data to ensure that all features are on a similar scale, which is important for machine learning algorithms.
- Splitting the data into training, validation, and test sets to evaluate the model's performance effectively.

4.3. Data Storage

The raw and preprocessed data are stored in a structured format using CSV files and DVC for version control. This allows us to track changes in the data over time and ensures reproducibility in our experiments. The data is stored in a dedicated S3 bucket for easy access and scalability (as shown in the tutorial https://mlops.swiss-ai-center.ch/).

5. ML Model Development

To develop the machine learning model for stock price prediction, we followed the https://mlops.swiss-ai-center.ch/ guide to set up our MLOps pipeline. This involved several key steps to ensure that the model is trained, validated, and deployed effectively.

5.1. Feature Engineering

TODO

5.2. Model Training and Validation

The model is trained using the preprocessed data, which includes the stock closing prices and other relevant features. We used LSTM (Long Short-Term Memory) neural networks, which are well-suited for time series data like stock prices. The training process involves optimizing the model's parameters to minimize the prediction error on the training set.

The validation set is used to tune hyperparameters and prevent overfitting.

TODO more details

5.3. Model Evaluation

The model's performance is evaluated using metrics such as Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R-squared (R^2). These metrics provide insights into how well the model

predicts stock prices compared to actual values. We also perform backtesting to assess the model's performance over historical data.

Here is a brief explanation of the evaluation metrics used:

- The Mean Absolute Error (MAE) measures the average absolute difference between predicted and actual values, showing how far predictions are from reality on average.
- The Root Mean Squared Error (RMSE) calculates the square root of the average squared errors, giving more weight to larger mistakes and highlighting significant prediction deviations.
- The R-squared (R^2) indicates how well the model explains the variance in the actual data, with values closer to 1 representing better predictive accuracy.

The best and worst predictions are visualized using plots to illustrate the model's performance and to better understand its strengths and weaknesses.

6. Deployment

To deploy the model, we use the https://mlops.swiss-ai-center.ch/ guidelines to set up a base environment for deployment.

The model is trained locally. On every pull request, the model is retrained in a Kubernetes pod using GitHub Actions. The trained model is then containerized using Docker and deployed to a cloud environment (e.g., AWS, GCP, or Azure) for scalability and accessibility. This setup correspond to the Chapter 3.7 of the MLOps guide.

TODO

6.1. CI/CD Pipeline

The CI/CD pipeline, defined in the MLOps workflow, automates the entire lifecycle of model development, testing, and deployment using GitHub Actions, DVC, CML, and BentoML. It runs on a self-hosted runner for training tasks and on GitHub-hosted runners for deployment.

The pipeline is triggered automatically when:

- Code is pushed to the main branch,
- A pull request is opened, or
- It is manually triggered via the GitHub Actions interface.

It consists of two main jobs:

1. Train and Report (train-and-report)

   Triggered on pull requests, this job automates model training and performance reporting:

   - Checks out the repository and sets up `Python 3.12`.
   - Installs dependencies and authenticates with Google Cloud.
   - Reproduces the model training pipeline using DVC (`dvc repro --pull`).
   - Pushes updated model artifacts and metrics to the DVC remote storage.
   - Automatically commits changes to `dvc.lock`.
   - Generates a CML report comparing the new experiment results to the main branch, including:
     - Parameter and metric differences (dvc params diff, dvc metrics diff),
     - Visual comparisons of plots (training history, prediction previews, and confusion matrices).
   - Publishes the report as a comment in the pull request for review.

2. Publish and Deploy (publish-and-deploy)

   Runs when changes are merged into the main branch. It performs the following steps:
   - Checks out the repository and installs dependencies.
   - Authenticates with Google Cloud and reproduces the final model pipeline using DVC.
   - Imports and builds the BentoML model artifact.
   - Containerizes the Bento model and pushes Docker images to the Google Container Registry.
   - Retrieves credentials for the Google Kubernetes Engine (GKE) cluster.
   - Updates the Kubernetes deployment manifest to reference the new image version (based on the Git SHA).
   - Applies the updated deployment and service configuration to deploy the new model version to Kubernetes.

6.2. Infrastructure

The infrastructure for deploying the machine learning model is set up using Google Cloud Platform (GCP). We utilize Google Kubernetes Engine (GKE) to manage the containerized application, ensuring scalability and reliability. The model is containerized using Docker, allowing for consistent deployment across different environments.

6.3. Scalability

As the model is deployed on GKE, it can easily scale based on demand. Kubernetes allows us to define resource limits and auto-scaling policies to handle varying workloads. This ensures that the application remains responsive even during peak usage times.

7. Monitoring

To monitor the model's performance, we choose to use only the pipeline as monitoring tool. This allows to track the model's performance over time and detect any potential issues related to data drift or model degradation, as the data used for training is updated weekly. However, this limits the monitoring capabilities compared to dedicated monitoring tools, which can monitor the model in real-time and provide more detailed insights into its performance.

7.1. Data Drift Detection

To detect data drift, we will implement the training of 2 models:

- A base model that will be trained once on the initial dataset and will not be updated.
- A fine-tuned model that will be retrained weekly using the new data from the previous week.

By comparing the performance of these two models over time, we can identify if data drift is occurring. If the fine-tuned model consistently outperforms the base model (with new data), it indicates that the underlying data distribution has changed, necessitating regular updates to maintain optimal performance.

TODO

7.2. Continuous Learning

To implement continuous learning, we will set up a weekly retraining schedule for the fine-tuned model. This involves:

- Collecting new data from the previous week.
- Preprocessing the new data to ensure it is in the correct format for training.
- Retraining the fine-tuned model using the updated dataset.
- Evaluating the performance of both the base and fine-tuned models on a validation set.

This process will be automated using the CI/CD pipeline, ensuring that the fine-tuned model is always up-to-date with the latest data.

TODO put in another section?

8. Conclusion

Assessment ??

TODO